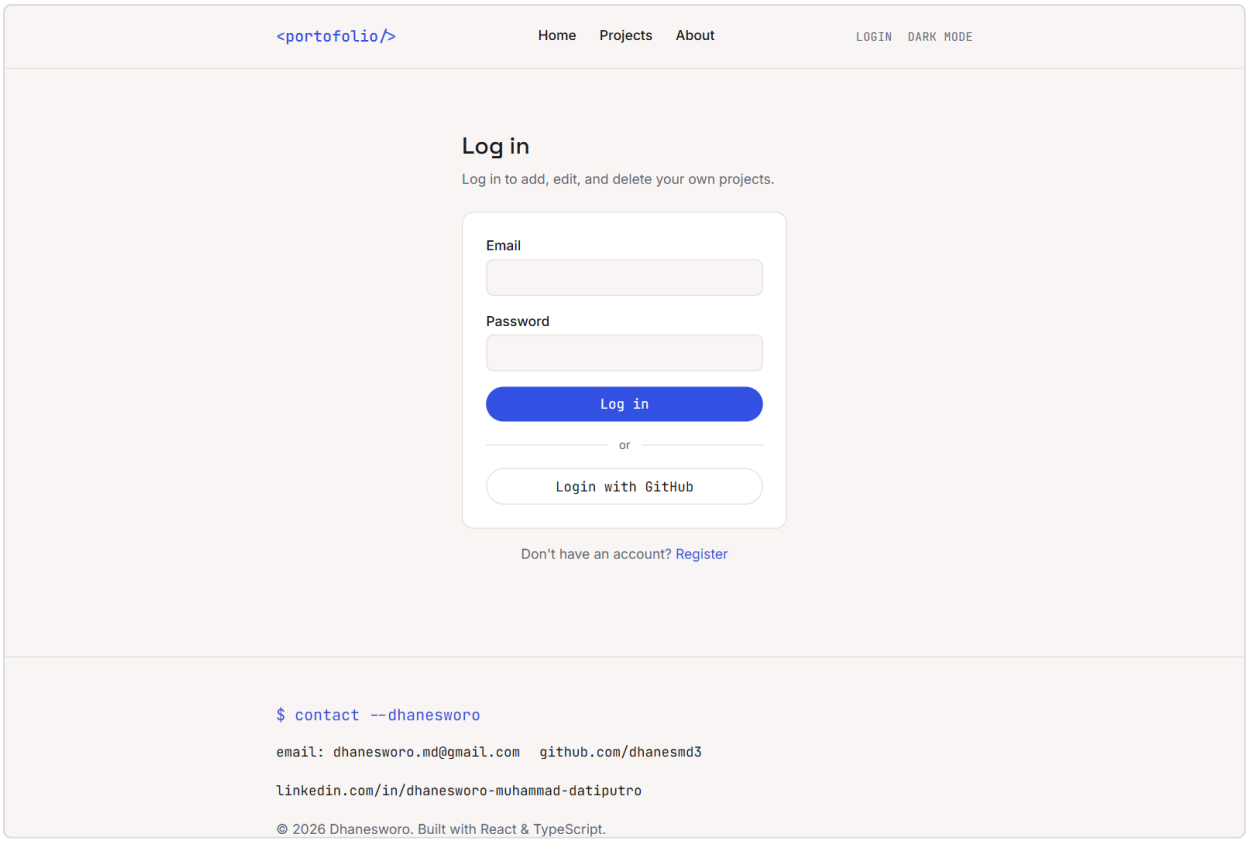
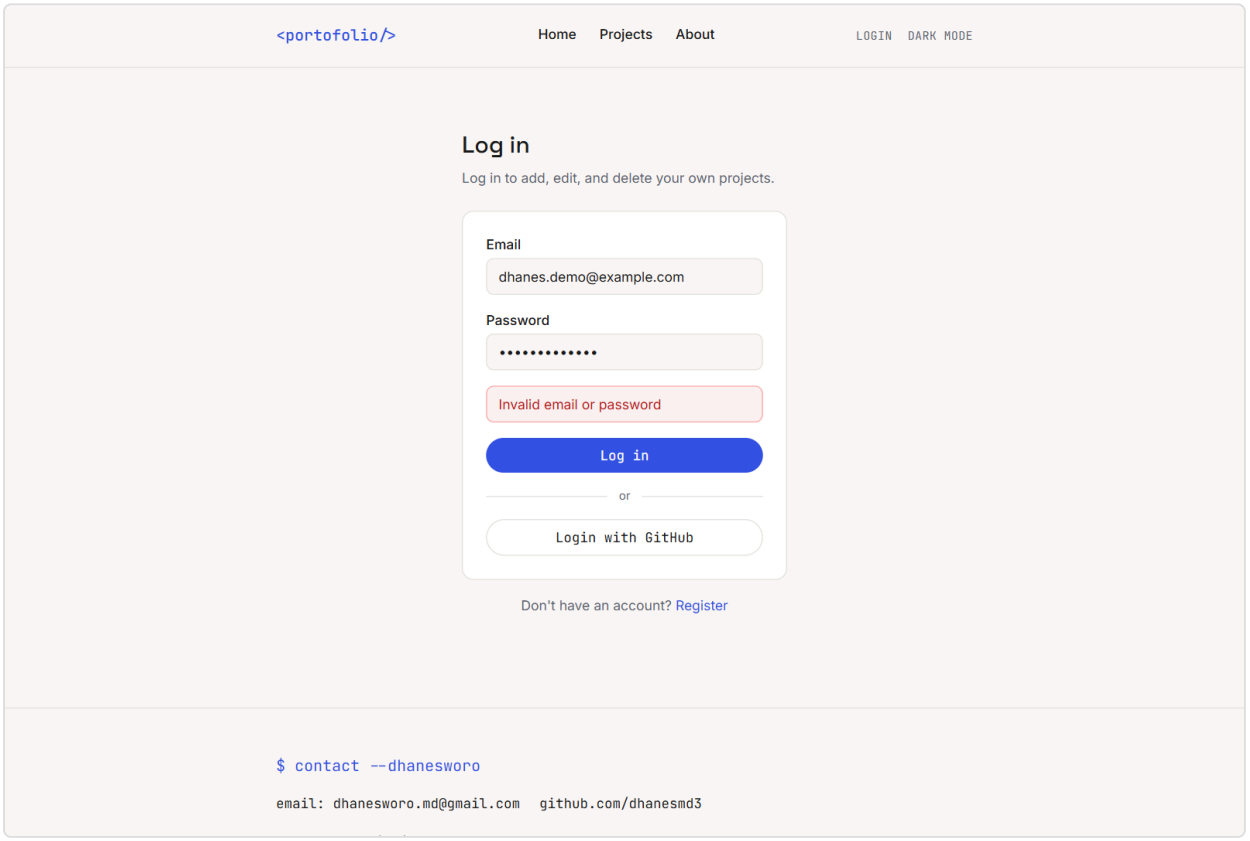
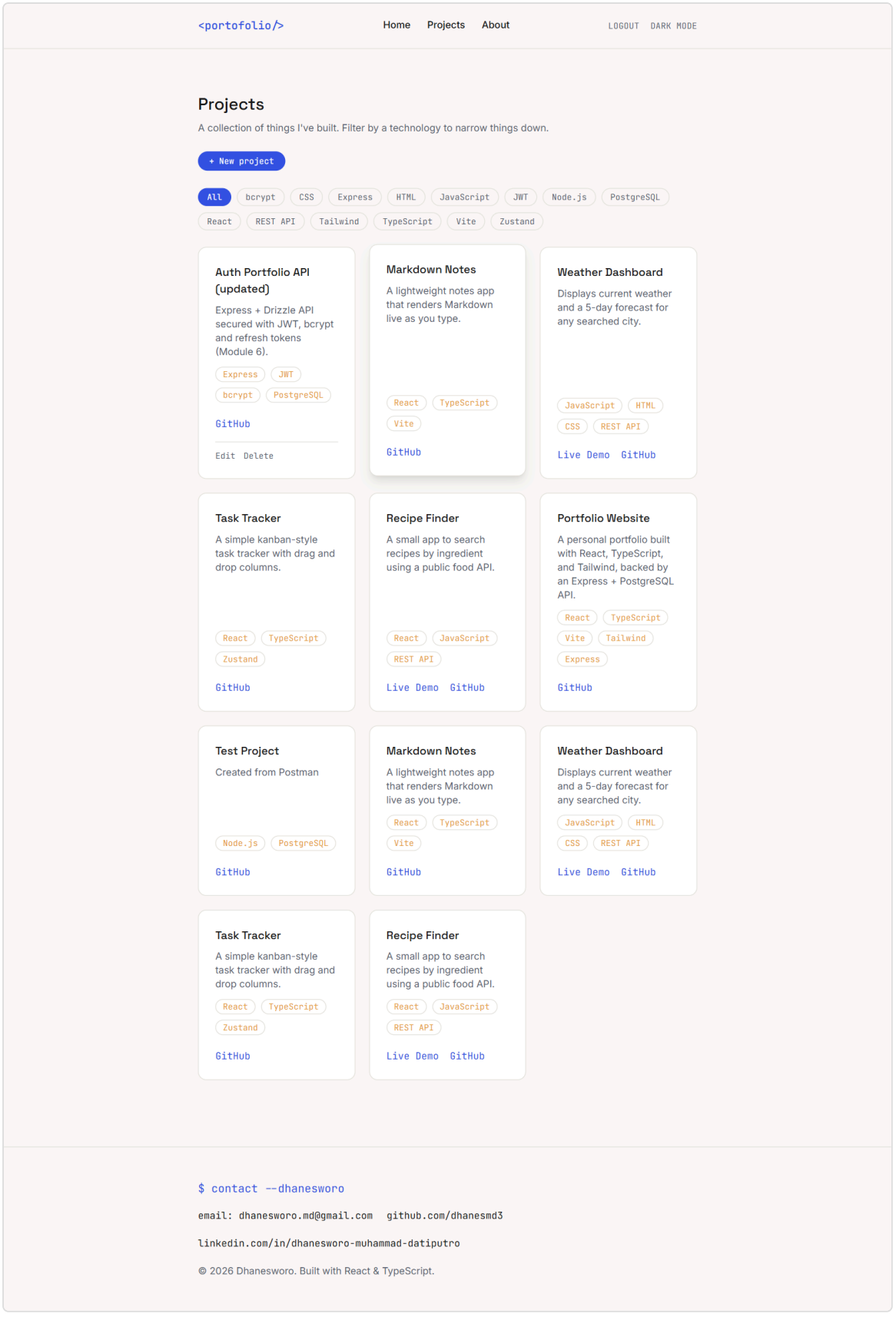
1. Login Form di Frontend



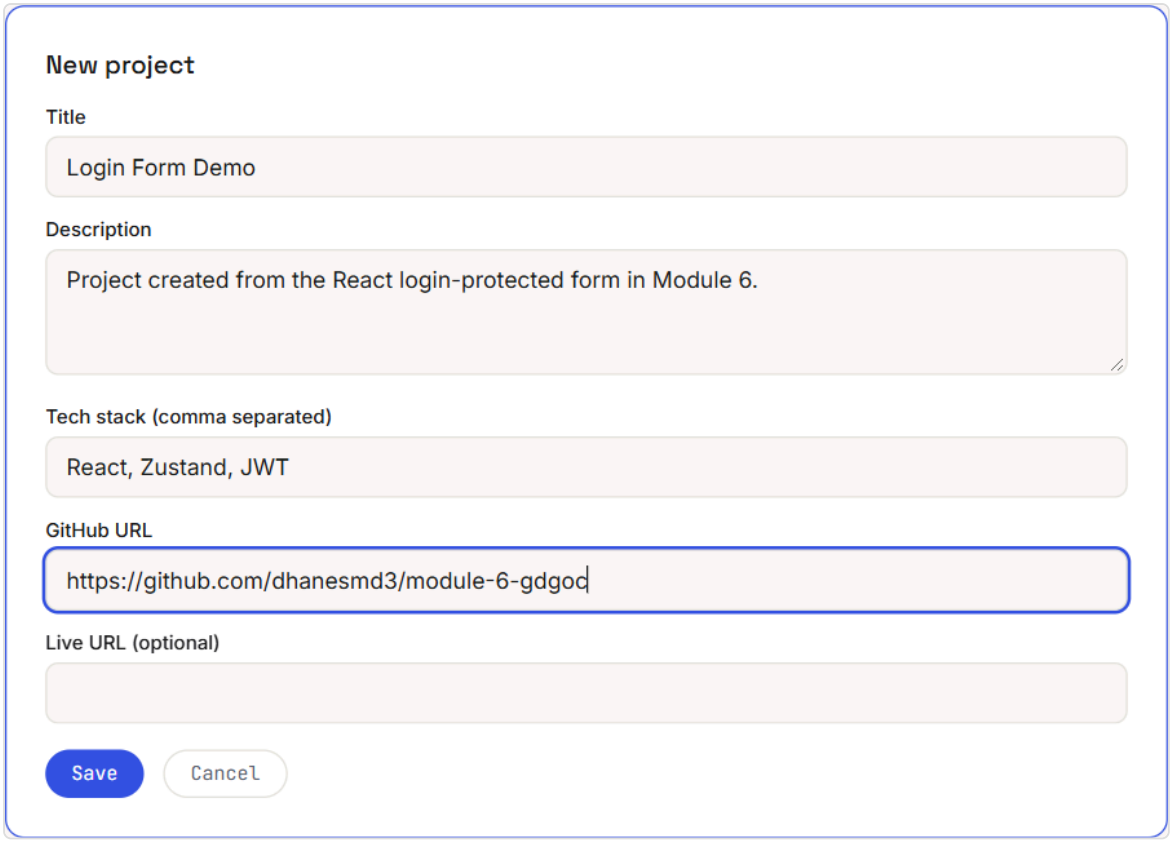
Gambar 1. Halaman login baru dengan form email dan password, serta tombol "Login with GitHub".



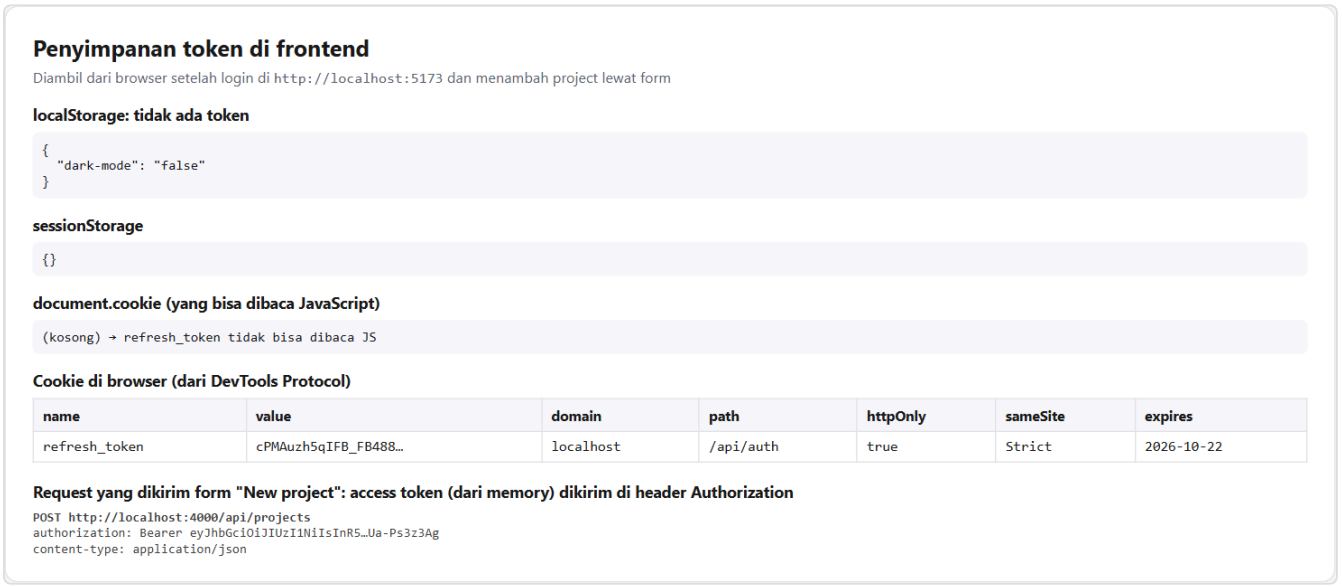
Gambar 2. Login dengan password salah ditolak backend dan menampilkan pesan "Invalid email or password".



Gambar 3. Setelah login, tombol "+ New project" muncul dan tombol Edit/Delete hanya tampil di project milik user yang sedang login.

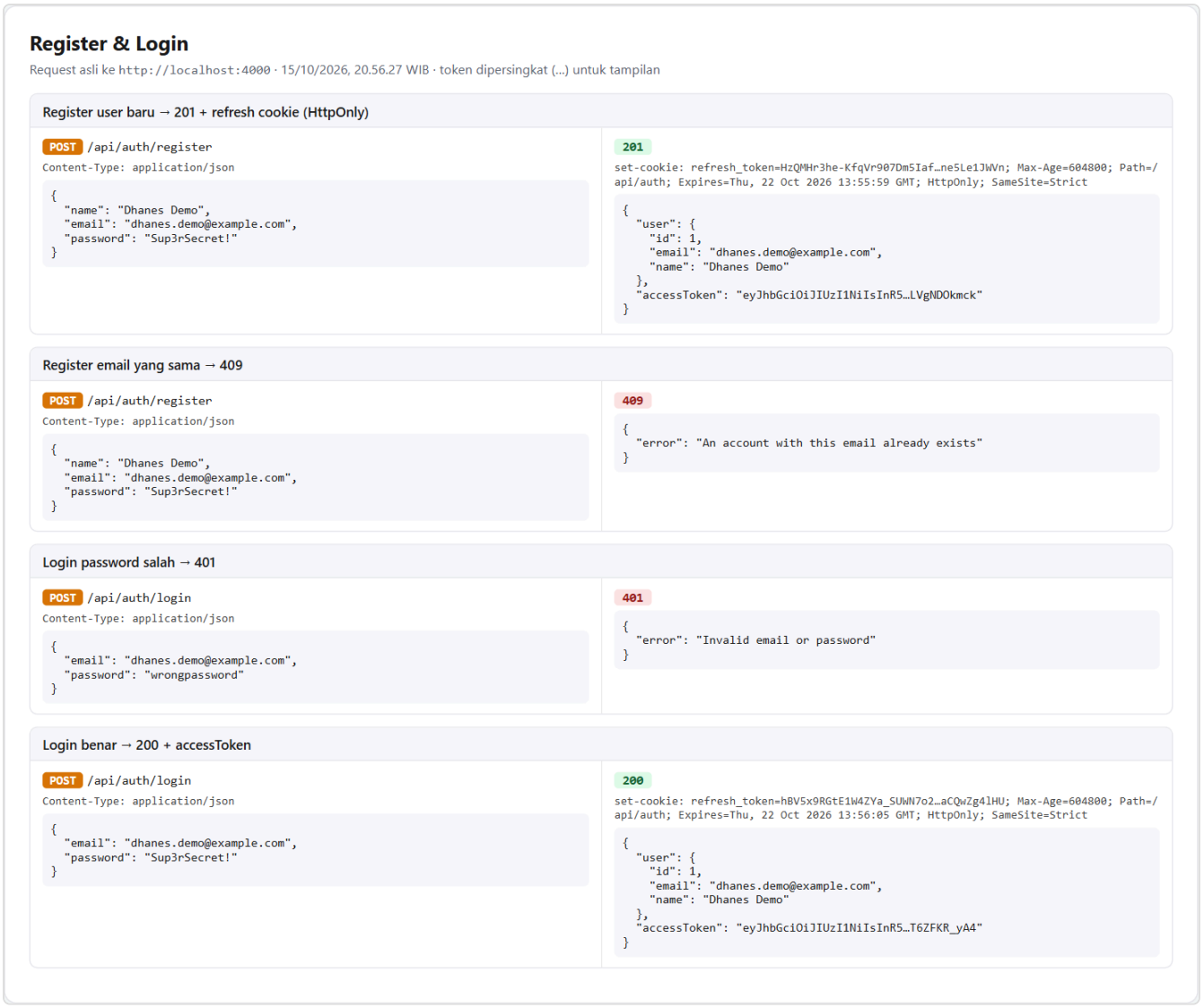


Gambar 4. Form tambah project yang hanya bisa diakses oleh user yang sudah login.

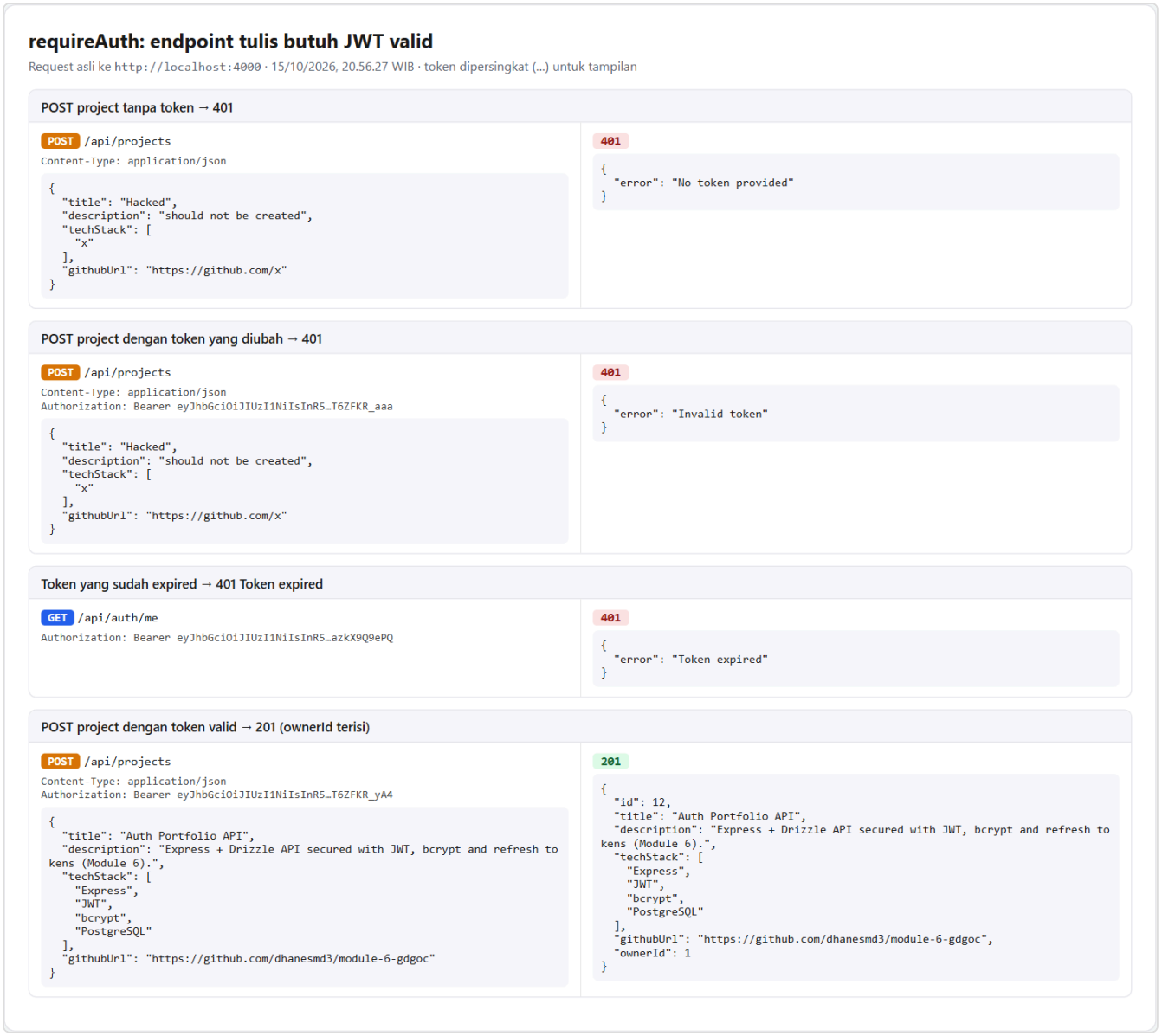


Gambar 5. Access token hanya disimpan di memory (tidak ada di localStorage), refresh token disimpan di cookie httpOnly, dan request dikirim dengan header Authorization: Bearer.

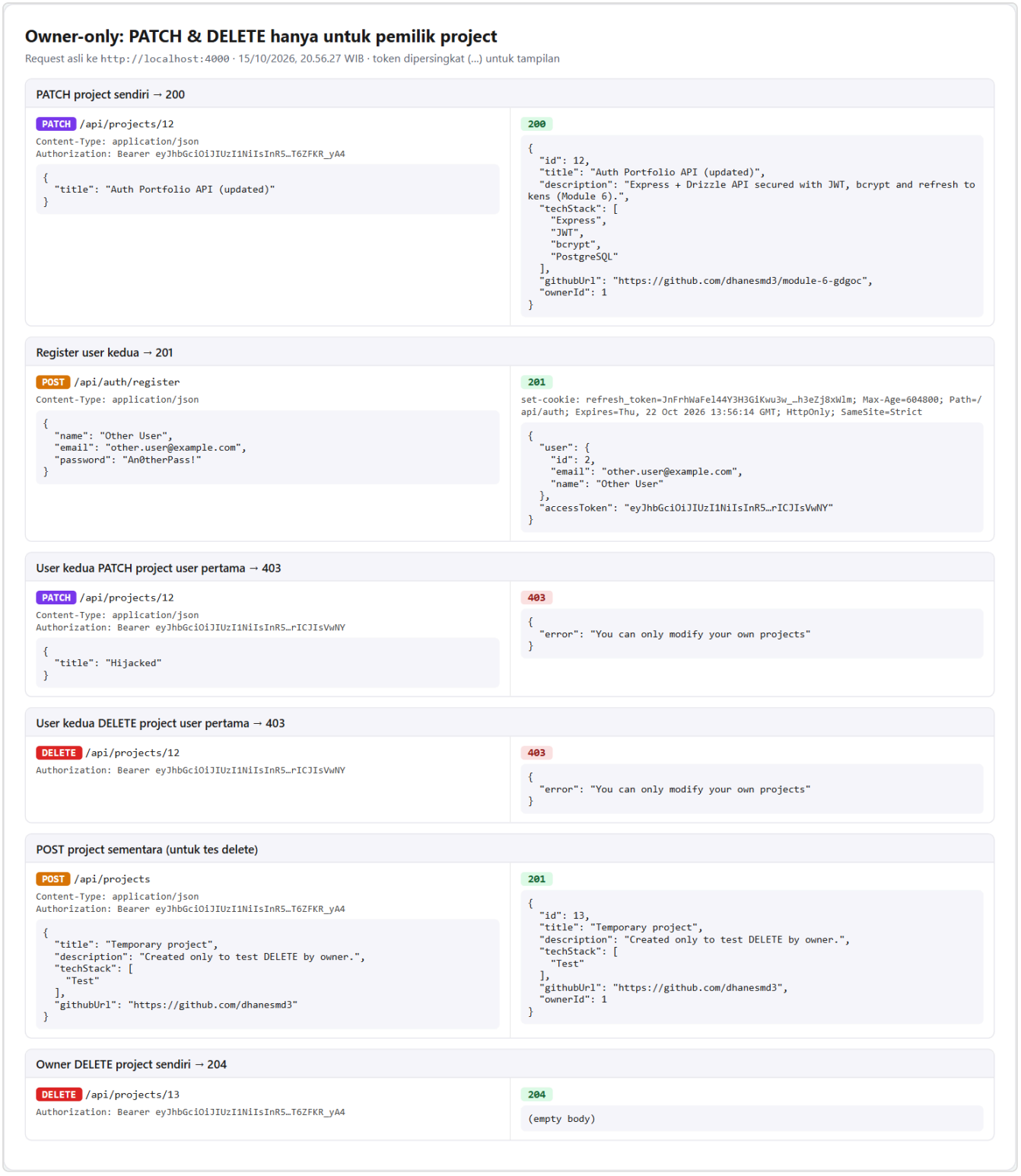
2. Endpoint Auth & Proteksi Backend (diuji dengan request asli)



Gambar 6. POST /api/auth/register menghasilkan 201, email ganda ditolak 409, password salah 401, dan POST /api/auth/login yang benar mengembalikan 200 beserta access token.



Gambar 7. Middleware requireAuth menolak request tanpa token, token yang dimodifikasi, dan token expired dengan status 401, serta menerima token yang valid.



Gambar 8. PATCH dan DELETE project hanya bisa dilakukan oleh pemiliknya, sedangkan user lain mendapat status 403.

3. JWT & Database

Isi JWT access token (hasil decode)

Token dari response `POST /api/auth/login` di atas. Struktur: header.payload.signature


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjEsImlhdCI6MTc5MjA3MjU2NSwiZXhwIjoxNzkyMDczNDY1fQ.vaP3zzAsagmIjDRgw77J6oDA8tkcGyJoXT6ZFKR_yA4
```


Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

```
{
  "userId": 1,
  "iat": 1792072565,
  "exp": 1792073465
}
```

iat = 2026-10-15T13:56:05.000Z · exp = 2026-10-15T14:11:05.000Z · **exp − iat = 900 detik = 15 menit** · payload hanya berisi `userId` (tanpa password/email)

Gambar 9. Hasil decode JWT: token ditandatangani dengan HS256, payload hanya berisi userId, dan masa berlakunya 15 menit.

Database (Supabase PostgreSQL)

Query langsung ke database setelah tes · 15/10/2026, 21.00.22 WIB · waktu dalam UTC

Tabel users: password disimpan sebagai hash bcrypt ($2b$12$...), bukan plaintext

id	email	name	password_hash	github_id	created_at
1	dhanes.demo@example.com	Dhanes Demo	$2b$12$ywADiF4784D9IY9STR5VJe2ohbfBpi3R2lb7IX9AtIB.8Cr5MeZ3W	NULL	2026-10-15 13:55:58
2	other.user@example.com	Other User	$2b$12$WQTZrUuXyhWpt1pJ3mBIIuccIds1k6dDEdewm9QqMfol7ID.E4wuG	NULL	2026-10-15 13:56:12

Tabel projects: kolom owner_id (project lama dari Module 5 = NULL)

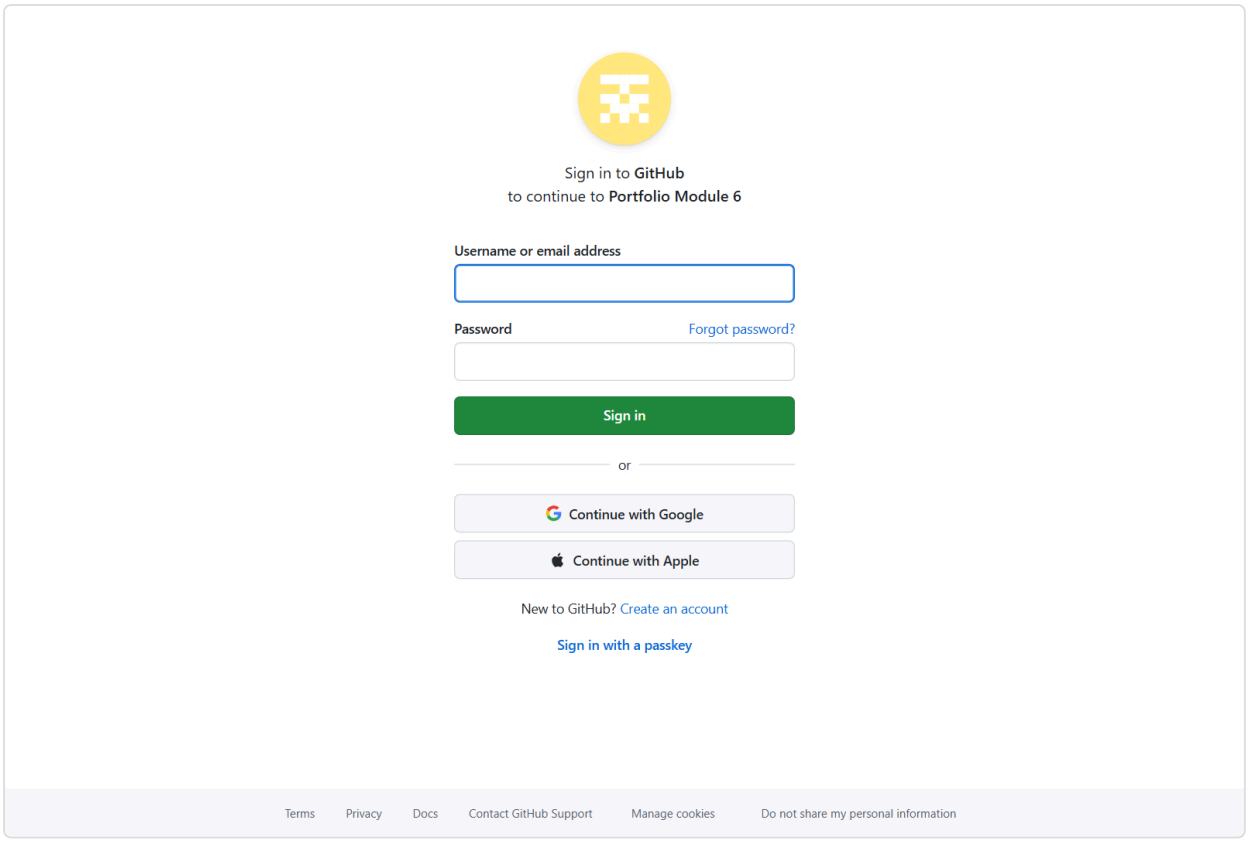
id	title	owner_id	deleted_at
14	Login Form Demo	1	NULL
13	Temporary project	1	2026-10-15 13:56:18
12	Auth Portfolio API (updated)	1	NULL
11	Markdown Notes	NULL	NULL
10	Weather Dashboard	NULL	NULL
9	Task Tracker	NULL	NULL
8	Recipe Finder	NULL	NULL

Tabel refresh_tokens: hanya hash SHA-256 yang disimpan; revoked_at terisi setelah rotate/logout; expires_at = +7 hari

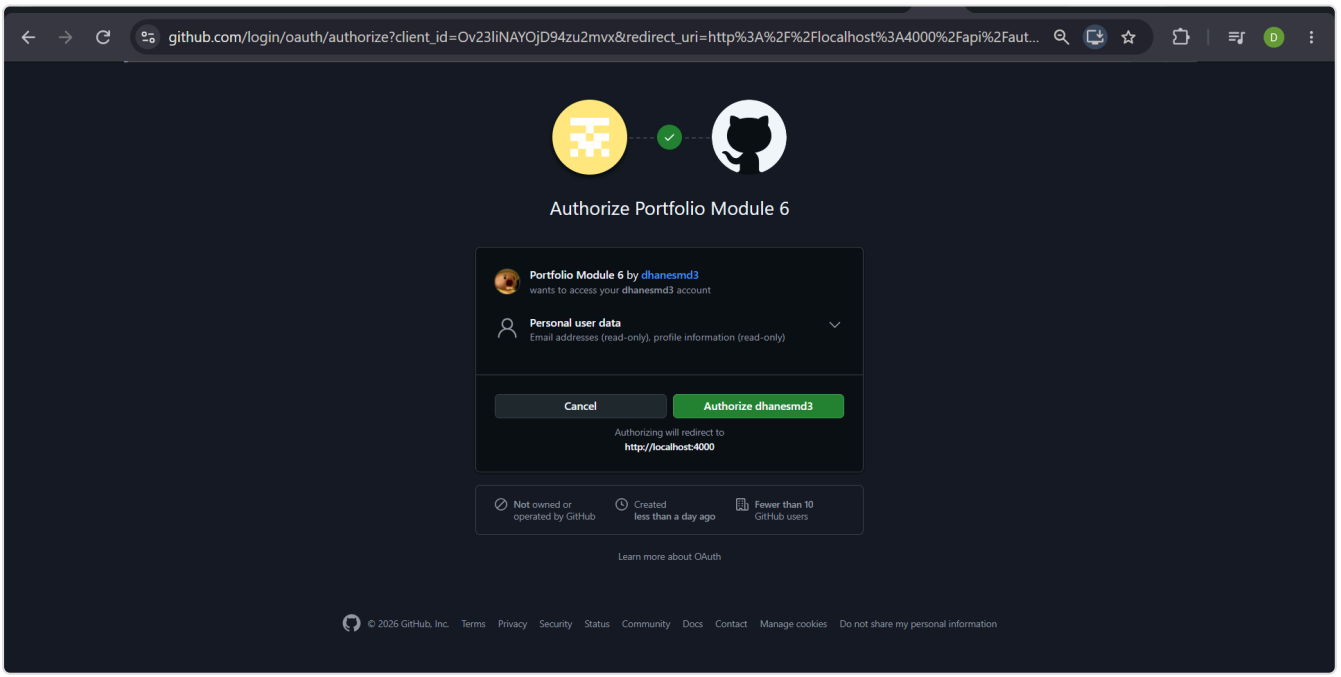
id	user_id	token_hash	expires_at	revoked_at
1	1	78c53cae9f57445c7afe…	2026-10-22 13:55:59	NULL
2	1	de3ced4b0989dd9116e4…	2026-10-22 13:56:04	2026-10-15 13:56:19
3	2	e0a062028c8244776713…	2026-10-22 13:56:13	NULL
4	1	e893848b18f1281db41c…	2026-10-22 13:56:20	2026-10-15 13:56:22
5	1	0f7dad90c5126f8ae78e…	2026-10-22 13:58:14	NULL

Gambar 10. Password disimpan sebagai hash bcrypt (bukan plaintext), project baru memiliki owner_id, dan refresh token disimpan dalam bentuk hash SHA-256.

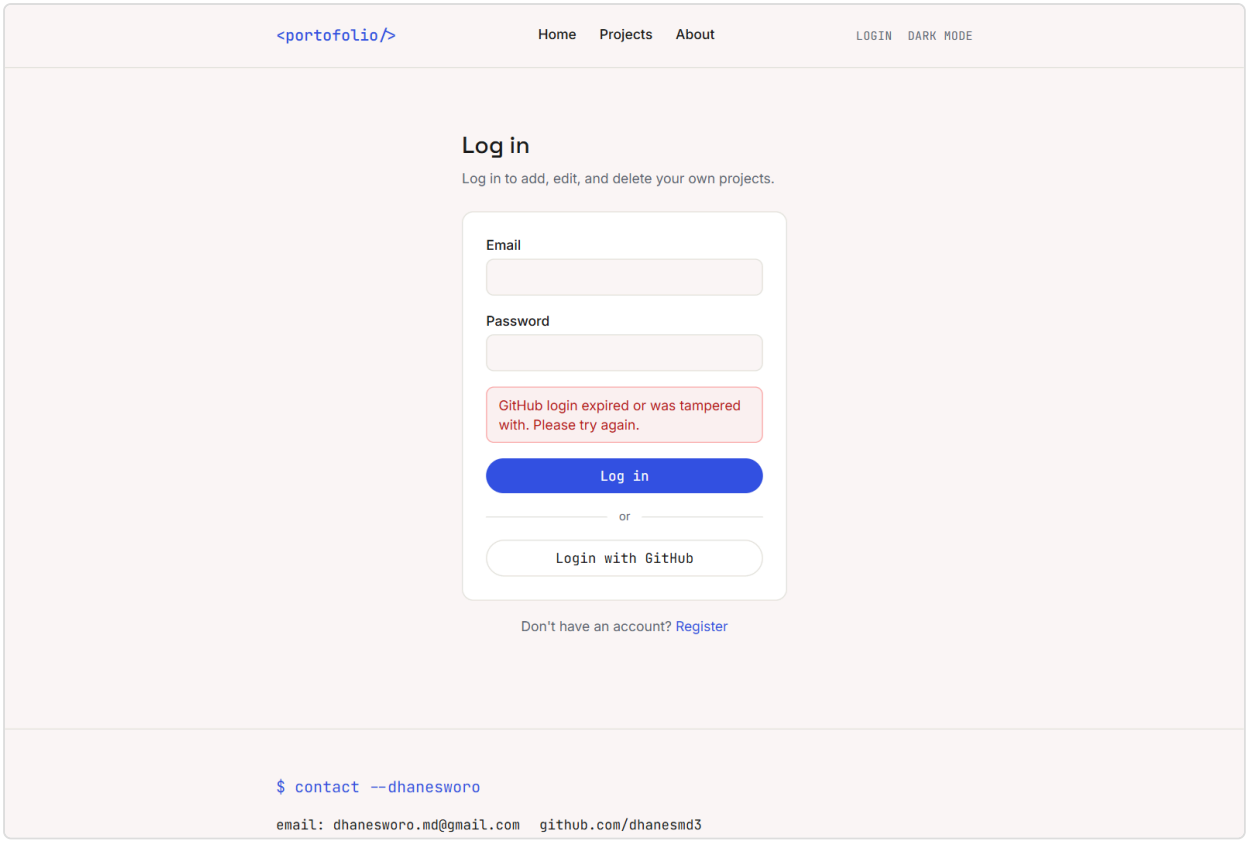
4. Bonus 1: Login with GitHub (OAuth 2.0)



Gambar 11. Tombol "Login with GitHub" mengarahkan user ke GitHub, yang mengenali OAuth App "Portfolio Module 6".



Gambar 12. Halaman consent GitHub meminta izin membaca profil dan email user, lalu akan redirect kembali ke http://localhost:4000.

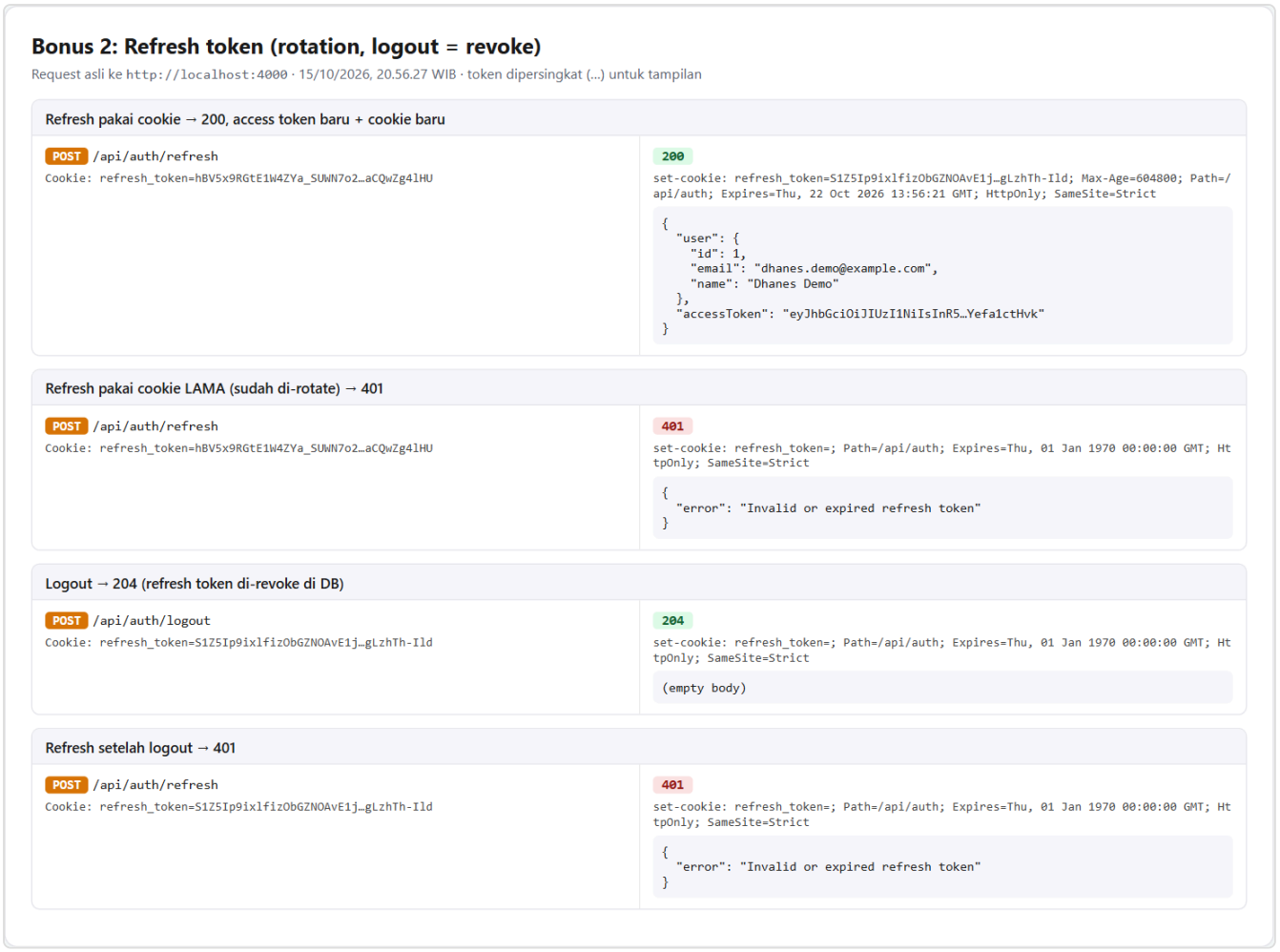


Gambar 13. Callback dengan parameter state palsu ditolak sebagai proteksi CSRF pada alur OAuth.

Alur OAuth 2.0 (Authorization Code):

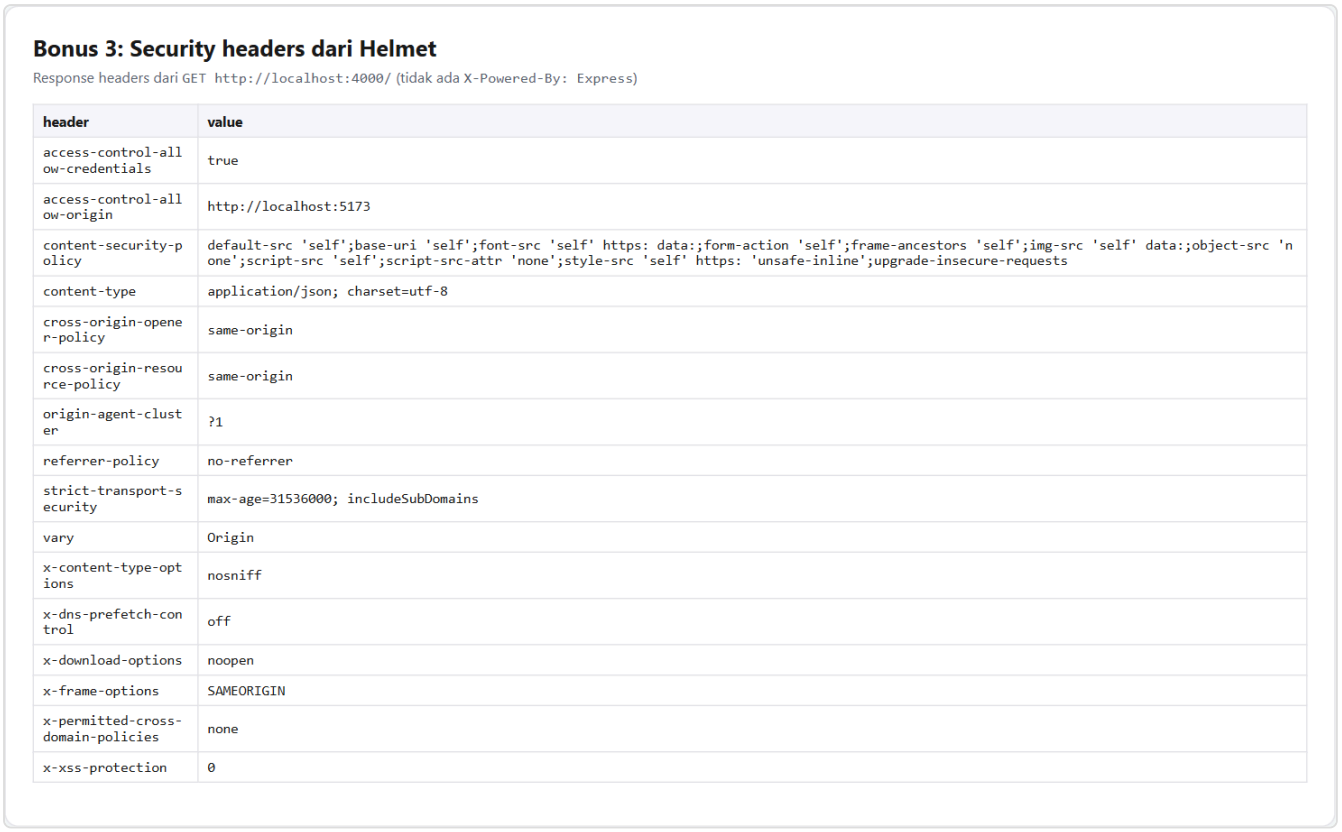
1. User klik "Login with GitHub" → `GET /api/auth/github`; backend membuat `state` acak, menyimpannya di cookie httpOnly, lalu redirect ke GitHub.
2. User menyetujui di halaman consent GitHub → GitHub redirect ke `/api/auth/github/callback?code=…&state=…`.
3. Backend mencocokkan `state` dengan cookie (anti-CSRF), lalu menukar `code` + `client_secret` menjadi access token GitHub (server-to-server).
4. Backend membaca profil dan email terverifikasi dari GitHub API, lalu mencari/membuat user dengan `github_id`.
5. Backend membuat sesi sendiri (refresh token di cookie httpOnly) dan redirect ke frontend, yang lalu memanggil `/api/auth/refresh` untuk mendapatkan JWT.

5. Bonus 2: Refresh Token



Gambar 14. Refresh token dirotasi setiap dipakai, cookie lama ditolak, dan logout me-revoke refresh token sehingga tidak bisa dipakai lagi.

6. Bonus 3: Security Headers (Helmet)



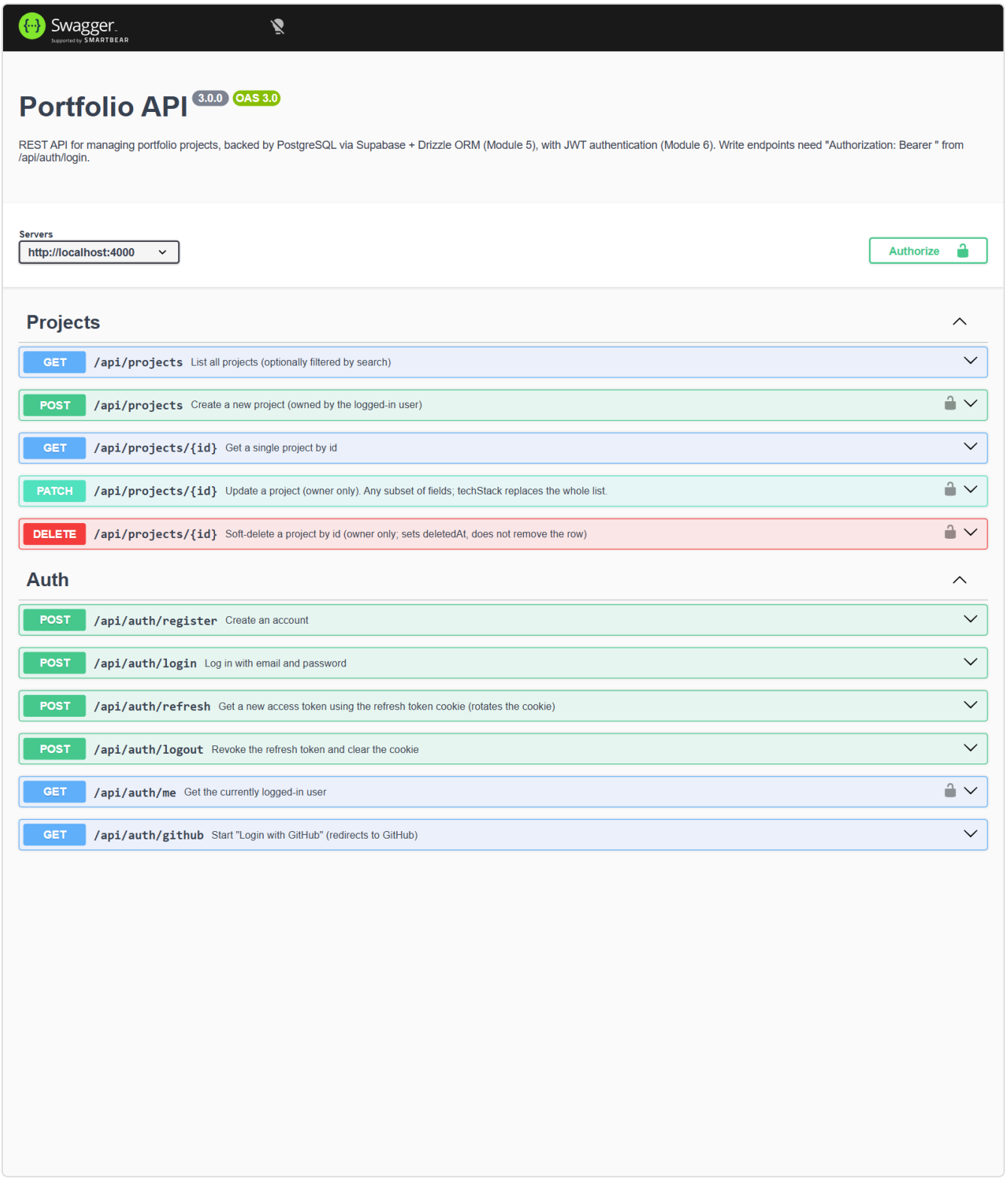
Bonus 3: Security headers dari Helmet

Response headers dari `GET http://localhost:4000/` (tidak ada `X-Powered-By: Express`)

header	value
access-control-allow-credentials	true
access-control-allow-origin	http://localhost:5173
content-security-policy	default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
content-type	application/json; charset=utf-8
cross-origin-opener-policy	same-origin
cross-origin-resource-policy	same-origin
origin-agent-cluster	?1
referrer-policy	no-referrer
strict-transport-security	max-age=31536000; includeSubDomains
vary	Origin
x-content-type-options	nosniff
x-dns-prefetch-control	off
x-download-options	noopen
x-frame-options	SAMEORIGIN
x-permitted-cross-domain-policies	none
x-xss-protection	0

Gambar 15. Helmet menambahkan security headers seperti Content-Security-Policy, HSTS, X-Frame-Options, dan X-Content-Type-Options.

7. Dokumentasi API



Gambar 16. Swagger menampilkan section Auth yang baru dan ikon gembok di endpoint yang membutuhkan token.